van Emde Boas trees solutions

20-E01 Bitmap successor

Derivation and check

One bits are at2,5,6. successor2=5,predecessor6=5,min=2. The leftmost-position convention removes bit-order ambiguity. A bitmap may locate values by word operations,but general scanning costs still depend on the word/universe model established in the lecture. A vEB successor above max is NIL.

20-E02 Cluster index

Derivation and check

high=floor(173/16)=10,low=173 mod16=13. It is local13 of cluster10 if not the independently stored minimum. Cached local extrema then determine same-group successor feasibility. A singleton{173} has min=max173 with all clusters/summary empty;indices alone do not mean the key is duplicated into a cluster.

20-E03 vEB recursion depth

Derivation and check

$\log_2$u follows32,16,8,4,2,1: six node calls and five recursive descents. This is $O(\log \log u)$,not32 or2^32. Two expensive calls per level would invalidate a depth-only time argument. Preallocated space remains Θ(u),not Θ(log log u).

20-E04 Updates with min excluded from clusters

Derivation and check

Initially min2,max15;clusters0 empty,1{1,2},2{1},3{0,3};summary{1,2,3}. insert1 swaps with min2,sets min1,and stores old2 as cluster0 local2;summary adds0. delete1 reads summary.min0 and cluster0.min2,promotes2,then removes local2. Empty cluster0 causes summary to delete0. delete15 removes local3 from cluster3,which still contains0,so summary retains3 and max becomes3・4+0=12. Final set{2,5,6,9,12}. Duplicating min2 in cluster0 would break the invariant linking summary to nonempty groups excluding the independent minimum.

van Emde Boas trees solutions (continued)

20-E05 Two calls need not mean two expensive recursions

Derivation and check

For empty-group insertion,empty-insert changes only min/max in O(1);the expensive call is summary. Nonempty-group insertion recurses only into cluster. A group becoming empty had one element,so its deletion used O(1) singleton base work before summary recursion. An expensive cluster deletion leaves the group nonempty and skips summary. Thus at most one expensive branch occurs per level. For evenly split $b=\log_2 u$,proto-minimum $S(b)=2S(b/2)+\Theta(1)=\Theta(b)$,or$\Theta(\log u)$. Proto-successor additionally computes a $\Theta(\log u)$ minimum,giving $S(b)=2S(b/2)+\Theta(b)=\Theta(b \log b)$,or$\Theta(\log u \log \log u)$. Square-root universes alone do not prove double-log time.

20-E06 Large-universe initialization must be counted

Derivation and check

Cluster decomposition yields u/2 universe-two bases. At≥4 bytes each,these alone require$2^{31} \cdot 4=2^{33}$ bytes=8 GiB,excluding summaries and other fields. This is a conservative model lower bound,not a memory measurement. vEB total time includes$\Theta(u)+O(m(1+\log \log u))$;a comparison tree starts empty in O(1),uses O(n) space,and costs $O(m \log(n+1))$ for operations. Initialization may dominate when m is modest. Lazy/hash alternatives need their own allocation and worst-case/expected/amortized proofs;single-operation logarithm comparisons omit decisive setup costs.

van Emde Boas trees solutions (continued)

Verification code appendix: veb_checks.py

```
"""Preallocated vEB implementation with explicit minimum exclusion invariant."""
from random import Random


class VEB:
    def __init__(self, universe):
        assert universe>=2 and universe&(universe-1)==0
        self.u=universe
        self.minimum=self.maximum=None
        if universe>2:
            bits=universe.bit_length()-1
            self.lower=1<<(bits//2)
            self.upper=1<<((bits+1)//2)
            self.summary=VEB(self.upper)
            self.cluster=[VEB(self.lower) for _ in range(self.upper)]

    def member(self,x):
        if self.minimum is None:
            return False
        if x in (self.minimum,self.maximum):
            return True
        return False if self.u==2 else self.cluster[x//self.lower].member(x%self.lower)

    def successor(self,x):
        if self.minimum is None or x>=self.maximum:
            return None
        if x<self.minimum:
            return self.minimum
        if self.u==2:
            return 1
        high,low=divmod(x,self.lower)
        cluster=self.cluster[high]
        if cluster.maximum is not None and low<cluster.maximum:
            return high*self.lower+cluster.successor(low)
        next_cluster=self.summary.successor(high)
        if next_cluster is None:
            return None
        return next_cluster*self.lower+self.cluster[next_cluster].minimum

    def predecessor(self,x):
        if self.minimum is None or x<=self.minimum:
            return None
        if x>self.maximum:
            return self.maximum
        if self.u==2:
            return 0
        high,low=divmod(x,self.lower)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

van Emde Boas trees solutions (continued)

```
        cluster=self.cluster[high]
        if cluster.minimum is not None and low>cluster.minimum:
            return high*self.lower+cluster.predecessor(low)
        previous=self.summary.predecessor(high)
        if previous is None:
            return self.minimum if x>self.minimum else None
        return previous*self.lower+self.cluster[previous].maximum

    def insert(self,x):
        assert 0<=x<self.u
        if self.member(x):
            return False
        self._insert(x)
        return True

    def _insert(self,x):
        if self.minimum is None:
            self.minimum=self.maximum=x
            return
        if x<self.minimum:
            x,self.minimum=self.minimum,x
        if self.u>2:
            high,low=divmod(x,self.lower)
            child=self.cluster[high]
            if child.minimum is None:
                self.summary._insert(high)
                child.minimum=child.maximum=low
            else:
                child._insert(low)
        if x>self.maximum:
            self.maximum=x

    def delete(self,x):
        assert 0<=x<self.u
        if not self.member(x):
            return False
        self._delete(x)
        return True

    def _delete(self,x):
        if self.minimum==self.maximum:
            self.minimum=self.maximum=None
            return
        if self.u==2:
            self.minimum=self.maximum=1 if x==0 else 0
            return
        if x==self.minimum:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

van Emde Boas trees solutions (continued)

```
            first=self.summary.minimum
            x=first*self.lower+self.cluster[first].minimum
            self.minimum=x
        high,low=divmod(x,self.lower)
        child=self.cluster[high]
        child._delete(low)
        if child.minimum is None:
            self.summary._delete(high)
            if x==self.maximum:
                last=self.summary.maximum
                self.maximum=self.minimum if last is None else
last*self.lower+self.cluster[last].maximum
        elif x==self.maximum:
            self.maximum=high*self.lower+child.maximum

    def validate(self):
        if self.u==2:
            assert (self.minimum is None)==(self.maximum is None)
            if self.minimum is None:
                return set()
            assert 0<=self.minimum<=self.maximum<2
            return {self.minimum,self.maximum}
        summary=self.summary.validate()
        values=set()
        nonempty=set()
        for high,child in enumerate(self.cluster):
            local=child.validate()
            if local:
                nonempty.add(high)
            values.update(high*self.lower+low for low in local)
        assert summary==nonempty
        if self.minimum is None:
            assert not values and self.maximum is None
            return set()
        assert self.minimum not in values
        values.add(self.minimum)
        assert min(values)==self.minimum and max(values)==self.maximum
        return values


def check_queries(tree, expected):
    assert tree.validate()==expected
    for x in range(tree.u):
        assert tree.member(x)==(x in expected)
        assert tree.successor(x)==min((y for y in expected if y>x),default=None)
        assert tree.predecessor(x)==max((y for y in expected if y<x),default=None)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

van Emde Boas trees solutions (continued)

```
def main():
    tree=VEB(16)
    for key in (2,5,6,9,12,15):
        tree.insert(key)
    check_queries(tree,{2,5,6,9,12,15})
    assert tree.summary.validate()=={1,2,3}
    assert tree.cluster[0].minimum is None
    assert (tree.successor(5),tree.successor(6),tree.predecessor(5))==(6,9,2)
    tree.insert(1)
    assert tree.cluster[0].validate()=={2}
    tree.delete(1)
    assert tree.cluster[0].minimum is None and tree.minimum==2
    tree.delete(15)
    assert tree.maximum==12
    for mask in range(256):
        tree=VEB(8)
        expected={x for x in range(8) if mask&(1<<x)}
        for x in sorted(expected,reverse=True):
            tree.insert(x)
        check_queries(tree,expected)
        for x in range(8):
            assert tree.delete(x)==(x in expected)
            expected.discard(x)
            check_queries(tree,expected)
    rng=Random(20)
    operations=0
    for universe in (2,4,8,16,32,64,256):
        tree=VEB(universe)
        expected=set()
        for _ in range(400):
            x=rng.randrange(universe)
            if rng.randrange(2):
                assert tree.insert(x)==(x not in expected)
                expected.add(x)
            else:
                assert tree.delete(x)==(x in expected)
                expected.discard(x)
            check_queries(tree,expected)
            operations+=1
    print(f"PASS vEB_all_u8_subsets=256 mixed_operations={operations} odd_bit_universes=8,32
min_exclusion_summary_and_all_queries=True")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

van Emde Boas trees solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study